
sqlbits
2026
sqlbits
22-25 APRIL - THE ICC WALES

Mastering Spark Notebooks and Capacity Optimization in Fabric

Just Blindbæk

Field CTO

Tabular Editor

just@justB.dk

/in/blindbaek

justB.dk

Slides

Microsoft Data Platform MVP

Conference organizor

Mastering Spark Notebooks and Capacity Optimization in Fabric

- How Spark Compute Works
- Capacity Consumption in Practice
- Optimizing Spark Notebooks
- Tips to Reduce CU Consumption
- Wrap-up & Q&A

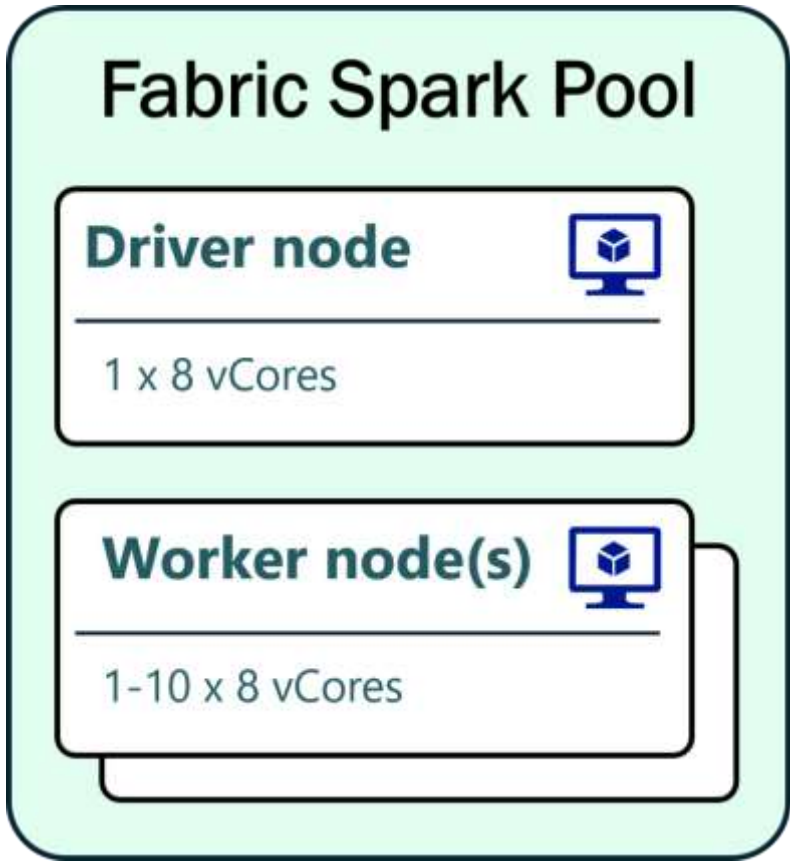
How Spark Compute Works

How Spark Compute Works

- Spark operates as a Platform-as-a-Service (PaaS) engine within the broader Software-as-a-Service (SaaS) ecosystem
- While it is serverless, you still “pay” for the compute resources as long as they remain allocated - even if you’re running lightweight operations that only use a fraction of the allocated power.
- Creating a Spark pool is free – you are only billed when your Spark job/notebook is active
- Your “server” in Fabric is a cluster of multiple machines (nodes), and in Fabric, this is called a Pool
  - Driver Node: The central coordinator that receives and manages user requests
  - Executor Nodes: The actual workers executing the tasks

Understanding the Default Starter Pool

- By default, Fabric assigns a Medium-sized Starter Pool, designed for fast startup times (within a few seconds)
  - Autoscaling: 1 to 10 nodes
  - Minimum allocation: 1 driver + 1 worker
  - Each node: 8 vCores
  - Minimum total allocation: 16 vCores (8 vCores x 2 nodes)



vCores mapping to Capacity Unites

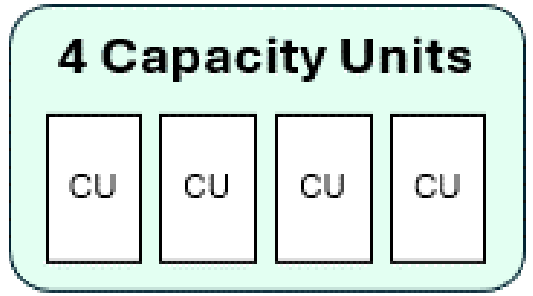
- 1 CU = 2 Spark vCores
- Starter Pool minimum total allocation: 16 vCores
- Then we need at least a F8 ?

Capacity SKU	Base CUs	Base Spark vCores
F2	2 CUs	4 vCores
F4	4 CUs	8 vCores
F8	8 CUs	16 vCores
Trial Capacity	64 CUs	128 vCores

Capacity Consumption in Practice

Spark Bursting Explained

- Allows temporary overuse of capacity
- Helps with concurrent workloads and the maximum cores available for a single Spark job
- Default is a 3x burst multiplier



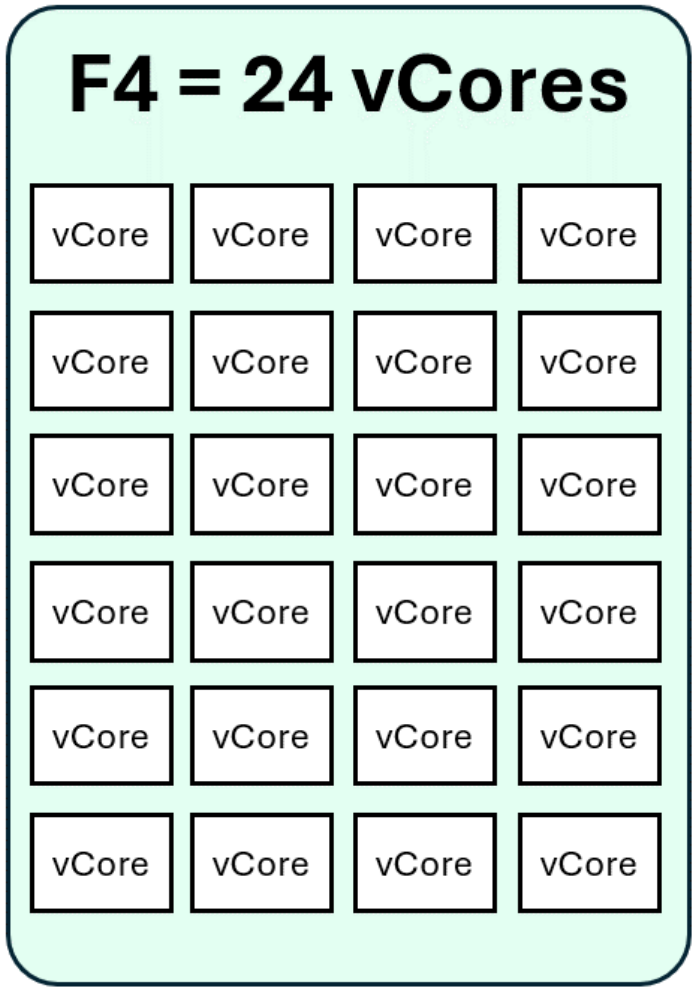
- Bursting Factor for F4 Capacity
  - 3x burst multiplier
  - 12 Capacity Units
  - 24 Spark vCores

Spark Bursting Factors and Max vCores

Capacity SKU	Base CUs	Base Spark vCores	Burst Multiplier	Max vCores with Bursting
F2	2 CUs	4 vCores	**5x**	**20 vCores**
F4	4 CUs	8 vCores	**3x**	**24 vCores**
F8	8 CUs	16 vCores	**3x**	**48 vCores**
Trial Capacity	64 CUs	128 vCores	**1x**	**128 vCores**

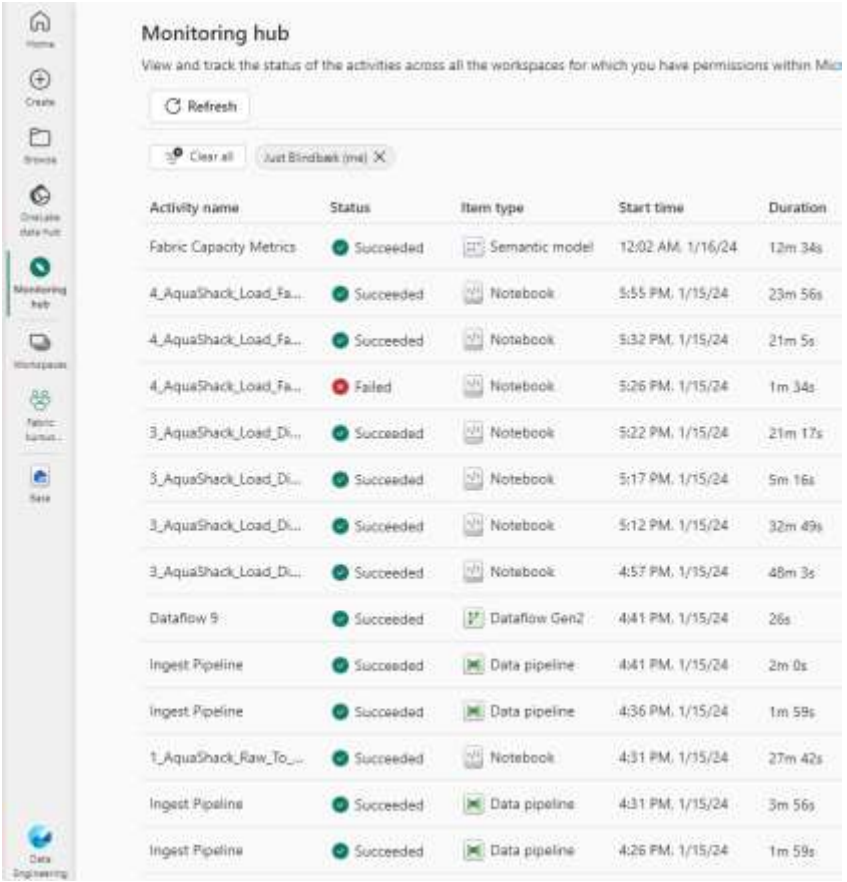
- Exceptions
  - Trial capacities do not offer bursting
  - F2 has a 5x burst factor, allowing it to scale from 4 vCores to 20 vCores
- It’s possible to disable job-level bursting

How Notebooks Use Capacity



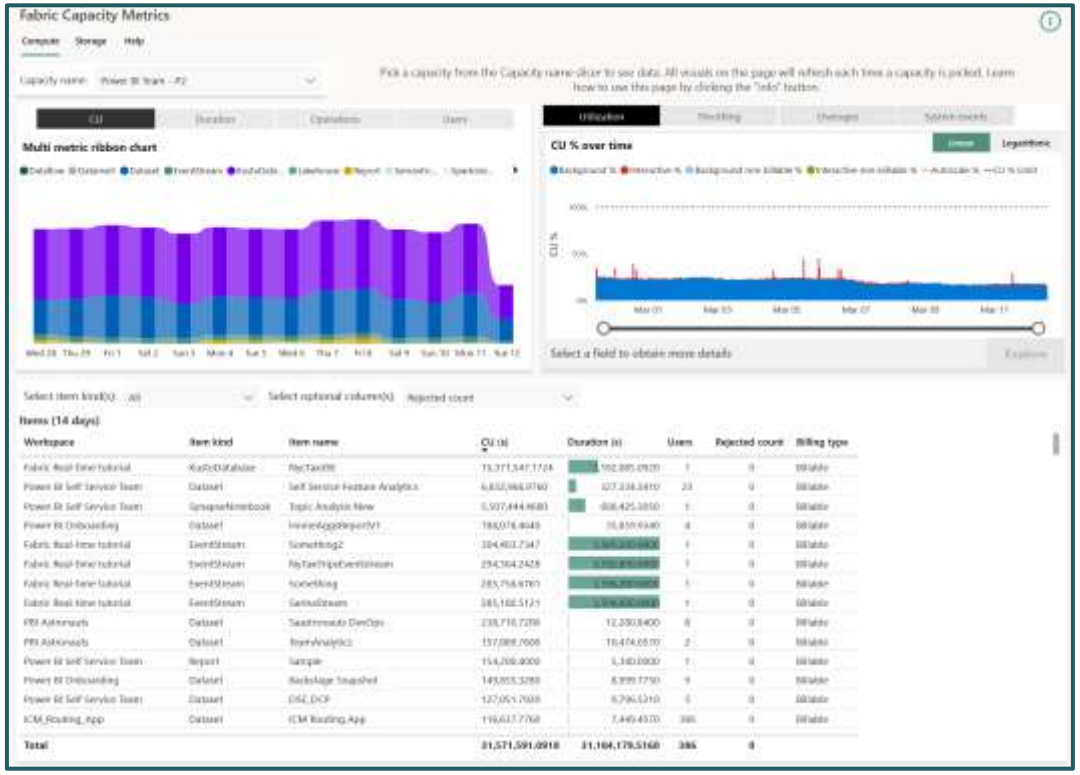
Monitor CU Consumption in the Monitor Hub

- Fabric provides built-in monitoring tools to help optimize Spark pool usage.
- Add Allocated Resources as a column in Monitor Hub to see how many executors and cores were used
- Go a step deeper and click on the activity name and then display the Spark configuration details, including
  - Number of executors over time
  - vCores used for drivers and workers
  - Execution breakdowns (duration, resource allocation, etc.)



Monitor CU Consumption in Capacity Metrics App

- Gain tenant-wide visibility into capacity usage across all Fabric workloads.
- Identify resource usage trends to optimize autoscale and mitigate throttling impacts.
- Compare preview and production workloads for data-driven capacity sizing decisions.
- Zoom in on workload operations and artifacts with detailed granularity down to 30 seconds.
- Monitor real-time operations and the impact of long-running jobs on capacity limits.
- Analyze user experience to plan efficient scale-ups and optimize resource allocation.



https://learn.microsoft.com/en-us/fabric/enterprise/metrics-app

Interactive vs. Scheduled Notebooks

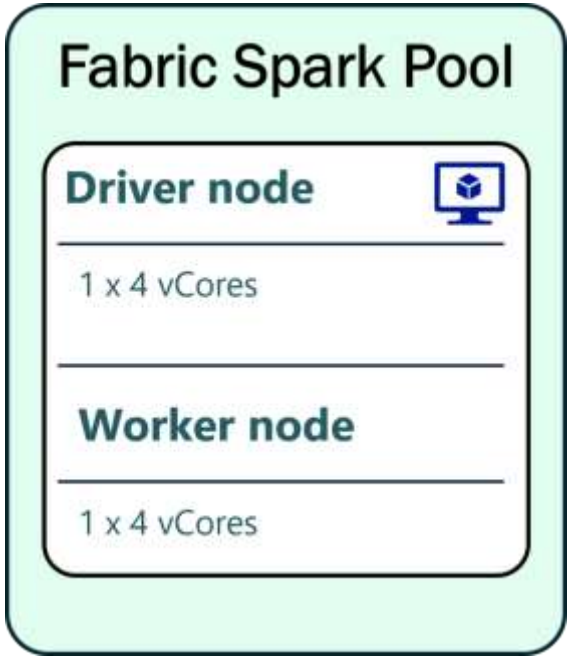
- Interactive:
  - Interactive notebooks do not queue - they return an error instead
  - Default idle timeout is 20 minutes. Change in workspace settings.
  - Configuring auto shut-down to prevent idle CU burn

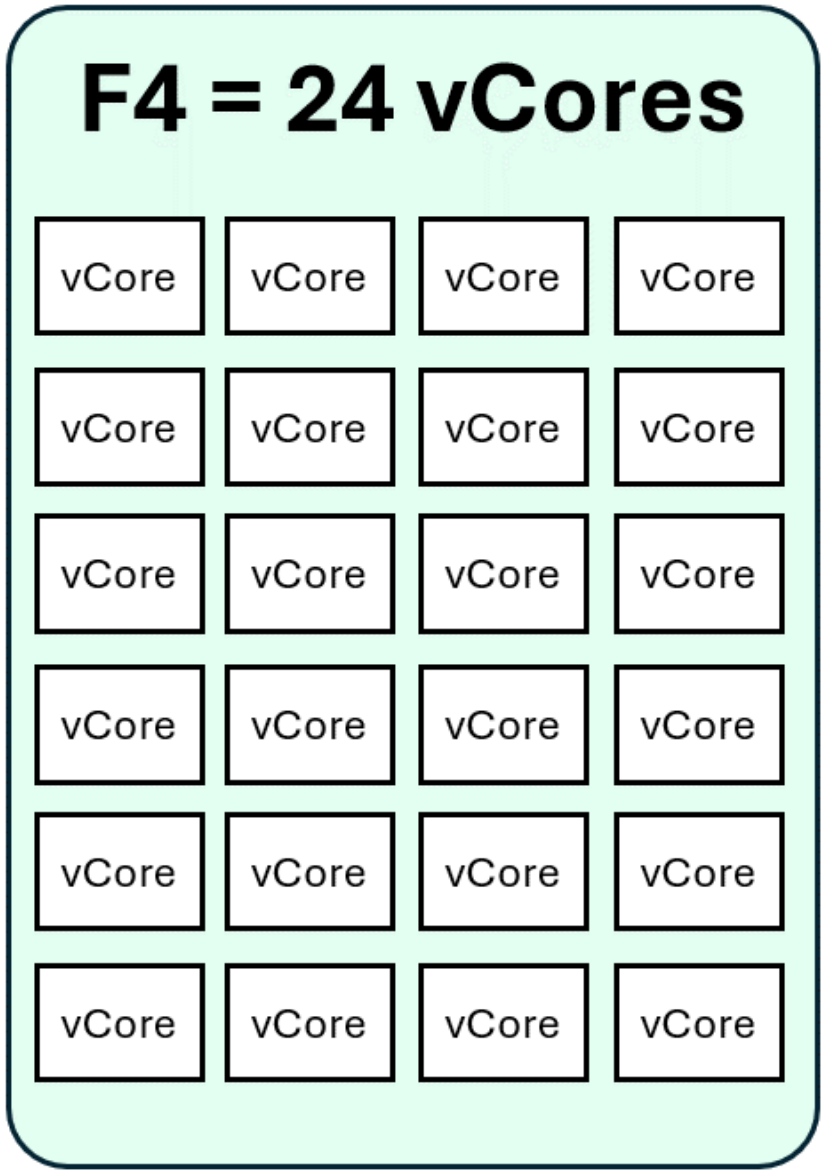
- Scheduled
  - Queue limit is equal to the Capacity SKU size
  - Jobs in the queue will retry automatically when resources become available
  - Automatically start and stop when the job completes, consuming CU only for the duration of execution.

Optimizing Pool Configurations

Single-node Spark Pool

- By setting autoscale to 1 for both minimum and maximum nodes, the driver and executor run on the same node, reducing the vCore requirement.
- Each session now uses only 8 vCores
- An F4 capacity can now support up to 3 active sessions instead of just 1

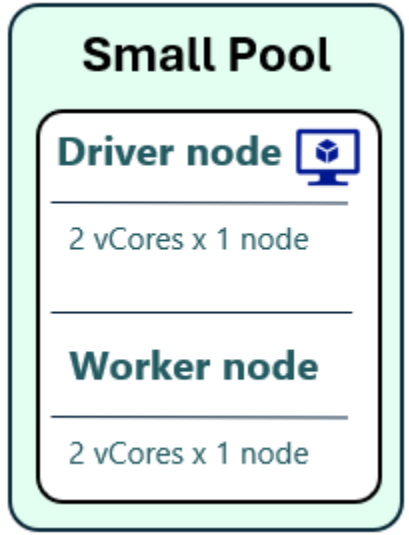



F4 = 24 vCores
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore

Custom Small Pool

- A Small Pool, forced to 1 node, uses only 4 vCores
- An F4 capacity can now run up to 6 active sessions

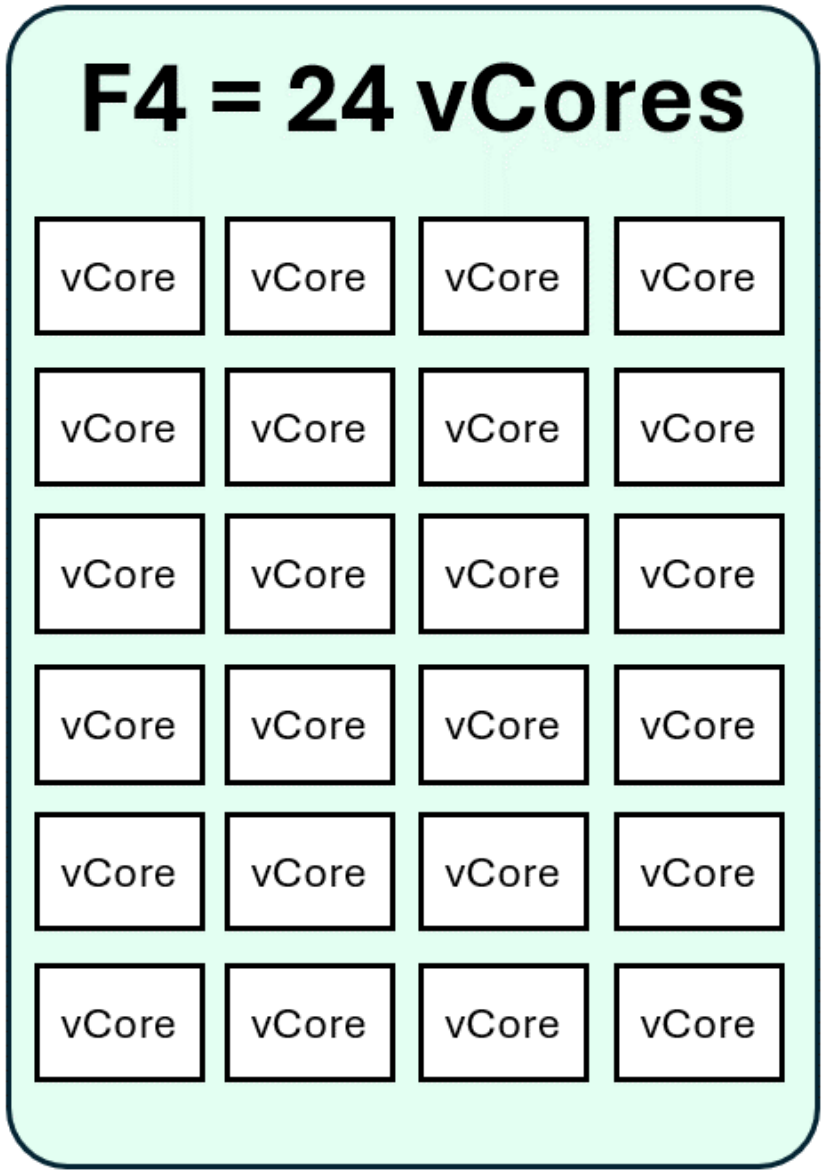
- Limited compute power per session
- Longer startup time compared to Starter Pools



Optimizing Session Sharing

High Concurrency Mode

- Multiple notebooks can share the same Spark session instead of launching separate compute clusters
- High Concurrency Mode is user-specific - sessions cannot be shared across different users.
- All notebooks must use the same default Lakehouse configuration.
- Enable High Concurrency Mode in Workspace Settings


F4 = 24 vCores
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore
vCore

When to use High Concurrency Mode

- Running multiple notebooks interactively – Reduces session startup time.
- Iterative development where you frequently test different scripts.

Use Case	Standard Session	High Concurrency Mode
Running a single notebook	✅ Best choice	❌ Not necessary
Multiple notebooks running concurrently	❌ Each creates its own Spark cluster (high CU usage)	✅ Notebooks share the same compute session
Interactivity & performance	⚠ Slower startup time for each new session	✅ Fast execution if a session is already active
CU optimization on limited capacity	❌ Higher CU burn	✅ More efficient CU usage

Using High Concurrency Mode in Pipelines

- Reduces redundant session startups, optimizing CU usage.
- Speeds up execution by attaching notebook activities to an existing session.
- Steps to configure session sharing in Pipelines
  - Add a Notebook activity to the Pipeline and select your notebook.
  - In Advanced settings, locate the Session tag field.
  - Assign a custom session tag (e.g., "pipeline_session").
  - Use the same session tag for all subsequent notebook activities that should share the session.

RunMultiple()

- Enables a parent notebook to trigger child notebooks within the same session
- Orchestrating multiple notebooks
- Execution follows a defined DAG (Directed Acyclic Graph) structure
- Defines dependencies between notebooks
- Enables parallel execution where possible

Tips to Reduce CU Consumption

Tips to Reduce Spark CU Consumption

- Single-node Spark Pools
- Lower auto-shutdown
- High Concurrency Mode for interactive work
- Monitor sessions
- RunMultiple for orchestration

Use Python Notebooks for Small Workloads

- Spark is a distributed system - overhead can be costly for simple tasks
- Try Python notebooks
  - Default configuration: 2 vCores
  - Lower CU consumption
  - Supports Polars and delta-rs for efficient data processing.

Autoscale Billing for Spark

- A flexible, pay-as-you-go billing model for Spark workloads in Microsoft Fabric.
- With this model enabled, Spark jobs use dedicated serverless resources instead of consuming compute from Fabric capacity.
- This serverless option optimizes cost and provides scalability without resource contention.
- The billing for Spark jobs is based solely on the compute used during job execution

Wrap-up: 3 takeaways

- Understand how Spark sessions consume capacity
- Optimize with Pool Configurations
- Optimize with Session Sharing & Orchestration

- Blogpost series:
  - https://justb.dk/blog/2025/02/fabric-spark-notebooks-and-cu-consumption/


sqlbits
2026
sqlbits
22-25 APRIL - THE ICC WALES